Zachariah King

July 28, 2025

Java Jackson JSON API

CSD 420-O301

Continuing on the topic of JSON or JavaScript Object Notation, which is a thin and light data-interchange format that is utilized primarily in web service APIs, this paper will dive deep into a specific API known as Jackson. As well as being a high performing API, this service is also open source so it is available to use and personalize freely. Its high performance status coms from its productivity, extensibility, and diverse usage in the Java landscape.

Jackson API was created by Tatu Saloranta in the year 2007 with the goal of being a high performing API that did not demand much, but still outshined in clean usability. Its main competitors that it wanted to take out were from JSON.org and GSON. After the years though, Jackson has developed into an organized collection of data-processing tools for Java. Not only does Jackson work with JSON, but also XML, YAML, and more!

The reason for Jackson's popularity lies in its feature and capabilities of course. These really create Jackson's characteristics of versatility and simplicity. Among these features and available actions, the most substantial ones have to be data binding, providing a Tree Model, a Streaming API, the use of annotations, polymorphic type handling, and custom serializing/ deserializing techniques. Data binding comes in either the simple method which maps JSON to Java primitives or the complex method with binds data using annotations for Java beans. Getting a little more graphic, the Tree Model is a tree-shaped build that gives way to more data access and modifications. The Streaming API is a little low-level API that allows for quick, memory-smart readings and writings that pertain to JSON. Annotations can be used to add more clarification to the serialization processes such as @JsonIgnore and @JsonInclude. The polymorphic type handling lets Jackson work with types of different compositions which

broadens its horizons greatly. The JSON serializing/deserializing interfaces allow for custom logic too. After all this, Jackson even mixes in with popular frameworks such as JAX-RS.

As stated above, the Jackson JSON API is made of modules and these include jackson-core (low-level streaming parser/generator), jackson-databind, jackson-annotations (standard Jackson annotations), jackson-module-kotlin (Kotlin support), and jackson-datatype-jsr310 (supports Java 8's Date & Time API). The modules here let Jackson help developers include only the necessary functions freeing up space for performance to really shine.

Jackson is best found through a ZIP file from GitHub releases that contains all the JARs needed. Jackson can also be acquired through Maven Central, but GitHub has the repository with the format asked in this assignment. The link to that GitHub page is: https://github.com/FasterXML/jackson-databind/releases .

The reason behind choosing the Java Jackson API is because of the massive support behind it that make it so reliable, efficient, and accessible. The libraries found in this API are relevant as well, not just reliable, such as the ones that make it possible for Jackson's data binding capabilities. Java developers definitely consider Jackson, if not choose it entirely, when needing an API. More benefits of this API include its community backing and real-time updates, so it is not difficult to see why this would be a top-choice.

Works Cited

FasterXML. (n.d.-a). *FasterXML/jackson: Main portal page for the jackson project*. GitHub. *https://github.com/FasterXML/jackson*

FasterXML. (n.d.-b). *Home*. GitHub. https://github.com/FasterXML/jackson-docs/wiki

Intro to the jackson objectmapper | baeldung. (n.d.-a). https://www.baeldung.com/jackson-object mapper-tutorial

Jackson Json Series | baeldung. (n.d.-b). https://www.baeldung.com/jackson